

Tugas 12: Principal Component Analysis (PCA)

Siti aisah -0110222129 ^{1*}

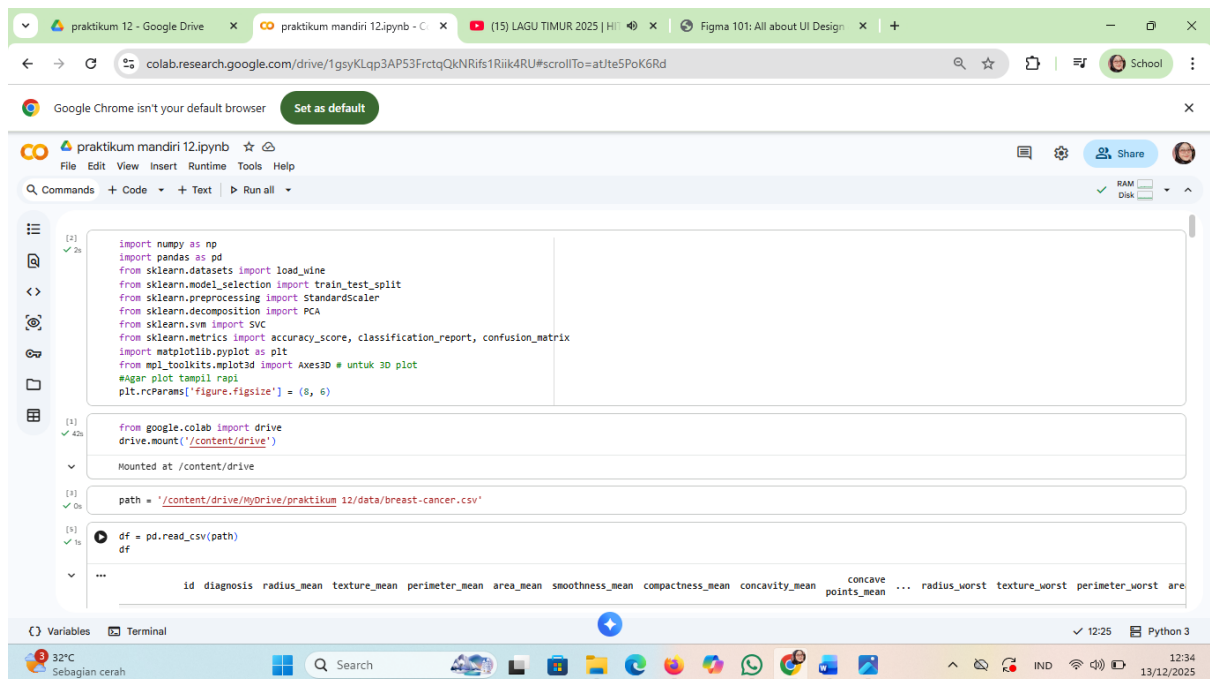
¹ Teknik Informatika, STT Terpadu Nurul Fikri, Depok

Abstract.

Machine Learning (Pembelajaran Mesin) adalah cabang dari Kecerdasan Buatan (AI) yang memungkinkan sistem untuk belajar secara mandiri dari data, mengenali pola, dan membuat keputusan atau prediksi tanpa diprogram secara **eksplisit** untuk setiap

Tugas mandiri 11

1. Perintah ini digunakan untuk memanggil library Pandas dengan alias pd. Pandas adalah pustaka Python yang berfungsi untuk mengolah, memanipulasi, dan menganalisis data, terutama dalam bentuk tabel (DataFrame).



```
[2] import numpy as np
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D # untuk 3D plot
#Agar plot tampil rapi
plt.rcParams['figure.figsize'] = (8, 6)

[1] from google.colab import drive
drive.mount('/content/drive')
Mounted at /content/drive

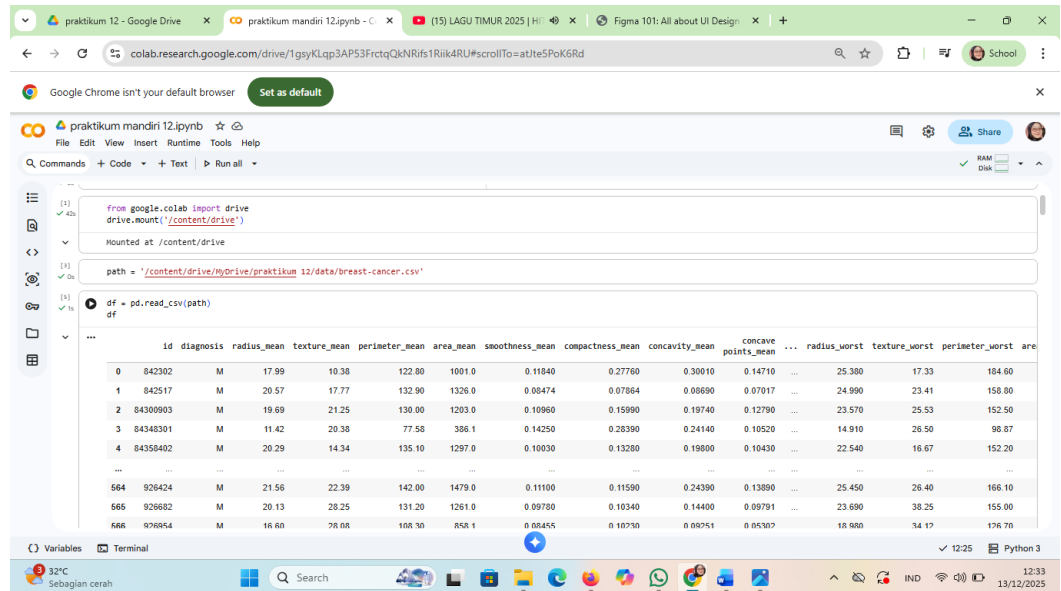
[3] path = '/content/drive/MyDrive/praktikum 12/data/breast-cancer.csv'

[4] df = pd.read_csv(path)
df

...
```

The screenshot shows a Google Colab notebook interface. The top bar includes the Google Drive link and the notebook name 'praktikum mandiri 12.ipynb'. The left sidebar shows the file explorer with a folder named 'praktikum 12'. The main area displays the code cells. The first cell imports various libraries including numpy, pandas, sklearn datasets, model selection, preprocessing, decomposition, SVM, metrics, and matplotlib. The second cell mounts the Google Drive. The third cell sets the file path to a CSV file in the drive. The fourth cell reads the CSV file into a DataFrame named 'df'. The bottom status bar shows the current time as 12:25 and the Python version as Python 3.

- Perintah ini digunakan untuk mengimpor modul drive dari Google Colab. Modul ini berfungsi untuk menghubungkan Google Drive dengan lingkungan kerja Google Colab.



The screenshot shows the Google Colab interface with the following code and output:

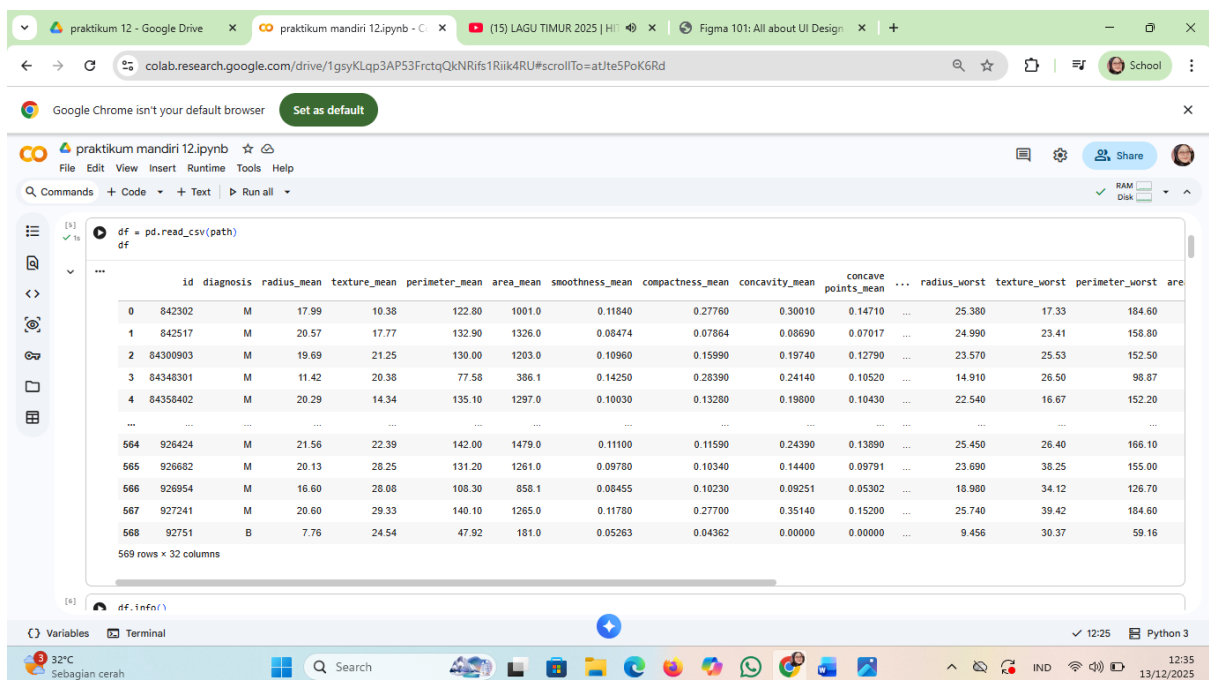
```
from google.colab import drive
drive.mount('/content/drive')

path = '/content/drive/MyDrive/praktikum 12/data/breast-cancer.csv'

df = pd.read_csv(path)
```

The output displays a DataFrame with the following columns: id, diagnosis, radius_mean, texture_mean, perimeter_mean, area_mean, smoothness_mean, compactness_mean, concavity_mean, concave points_mean, radius_worst, texture_worst, perimeter_worst, area_worst. The DataFrame contains 569 rows of data.

- proses membaca dataset CSV dari Google Drive dan menampilkannya menggunakan Pandas.



The screenshot shows the Google Colab interface with the following code and output:

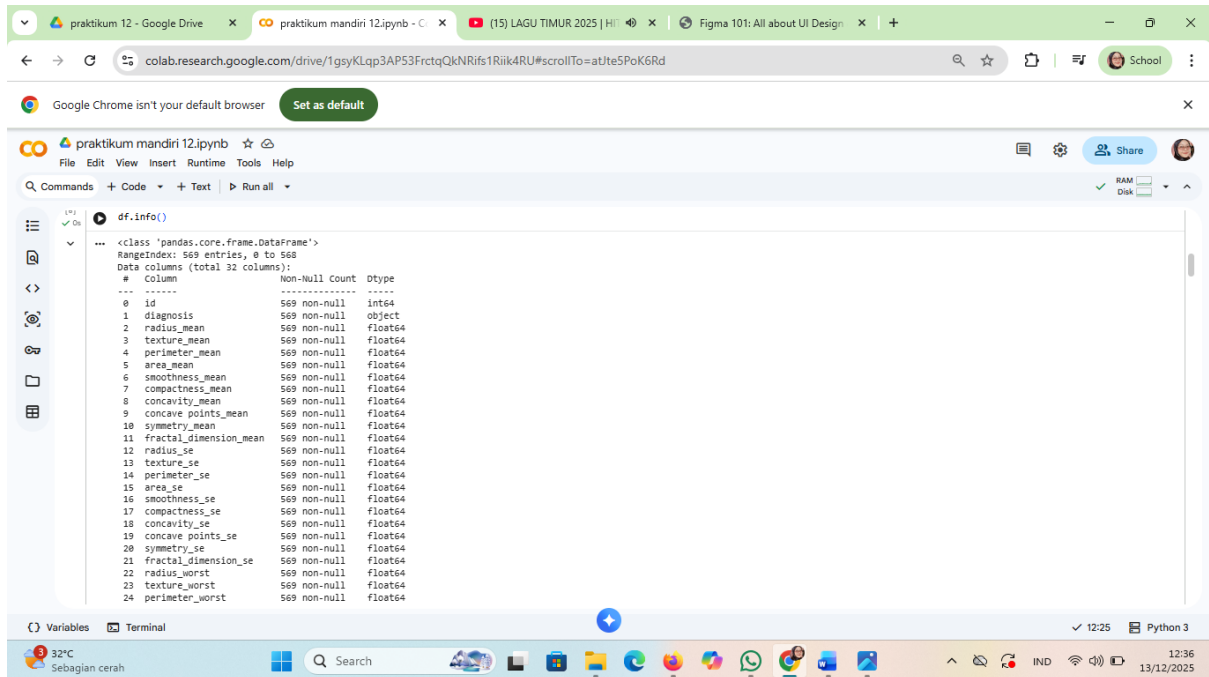
```
df = pd.read_csv(path)
```

The output displays the first few rows of the DataFrame:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	radius_worst	texture_worst	perimeter_worst	area_worst
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.30010	0.14710	25.380	17.33	184.60	158.80
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.08690	0.07017	24.990	23.41	158.80	152.50
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.19740	0.12790	23.570	25.53	152.50	98.87
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.24140	0.10520	14.910	26.50	152.20	176.70
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.19800	0.10430	22.540	16.67	152.20	166.10

The DataFrame contains 569 rows and 15 columns.

4. digunakan untuk mendapatkan ringkasan informasi tentang DataFrame.

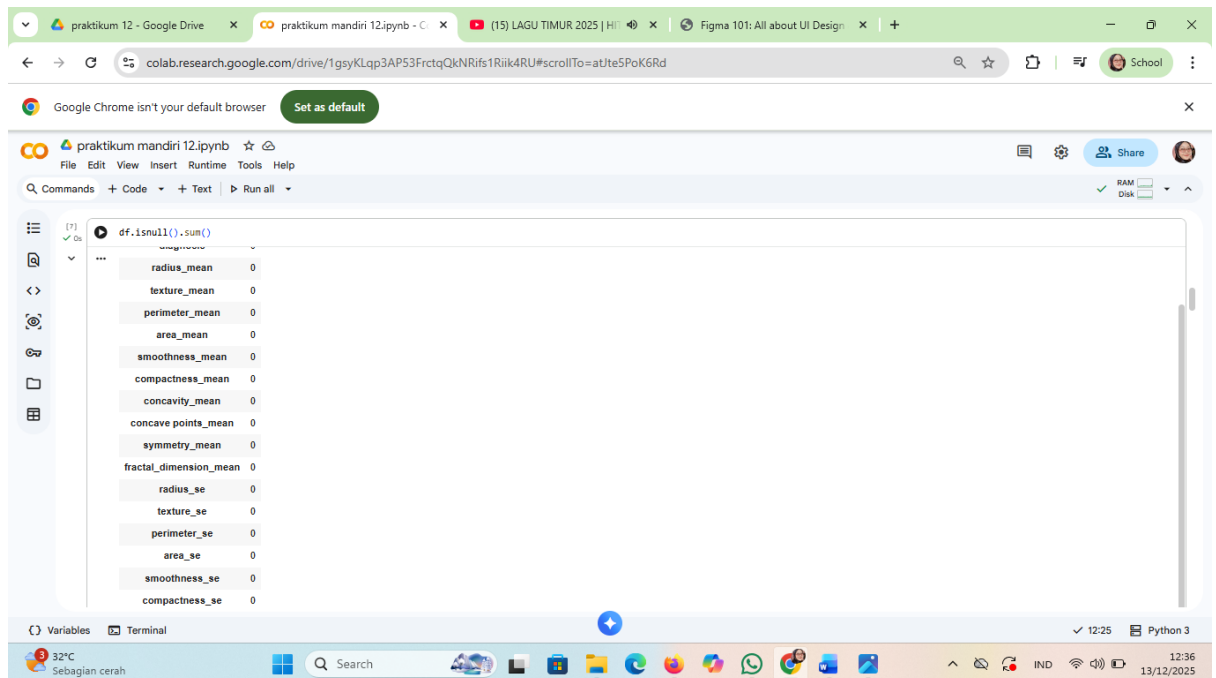


The screenshot shows a Google Colab notebook interface. The browser tabs at the top include 'praktikum 12 - Google Drive', 'praktikum mandiri 12.ipynb', '(15) LAGU TIMUR 2025 | HIT', and 'Figma 101: All about UI Design'. The address bar shows a Google Drive link. The notebook title is 'praktikum mandiri 12.ipynb'. The code cell contains the command `df.info()`. The output shows the DataFrame's structure:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 32 columns):
#   Column                                     Non-Null Count  Dtype
---  ---                                     -
0   id                                         569 non-null    int64
1   diagnosis                                 569 non-null    object
2   radius_mean                              569 non-null    float64
3   texture_mean                             569 non-null    float64
4   perimeter_mean                           569 non-null    float64
5   area_mean                                569 non-null    float64
6   smoothness_mean                          569 non-null    float64
7   compactness_mean                         569 non-null    float64
8   concavity_mean                           569 non-null    float64
9   concave points_mean                      569 non-null    float64
10  symmetry_mean                             569 non-null    float64
11  fractal_dimension_mean                   569 non-null    float64
12  radius_se                                569 non-null    float64
13  texture_se                                569 non-null    float64
14  perimeter_se                              569 non-null    float64
15  area_se                                   569 non-null    float64
16  smoothness_se                            569 non-null    float64
17  compactness_se                           569 non-null    float64
18  concavity_se                             569 non-null    float64
19  concave points_se                        569 non-null    float64
20  symmetry_se                              569 non-null    float64
21  fractal_dimension_se                     569 non-null    float64
22  radius_worst                             569 non-null    float64
23  texture_worst                            569 non-null    float64
24  perimeter_worst                          569 non-null    float64
```

The bottom of the screen shows a Windows taskbar with the date 13/12/2025 and time 12:36.

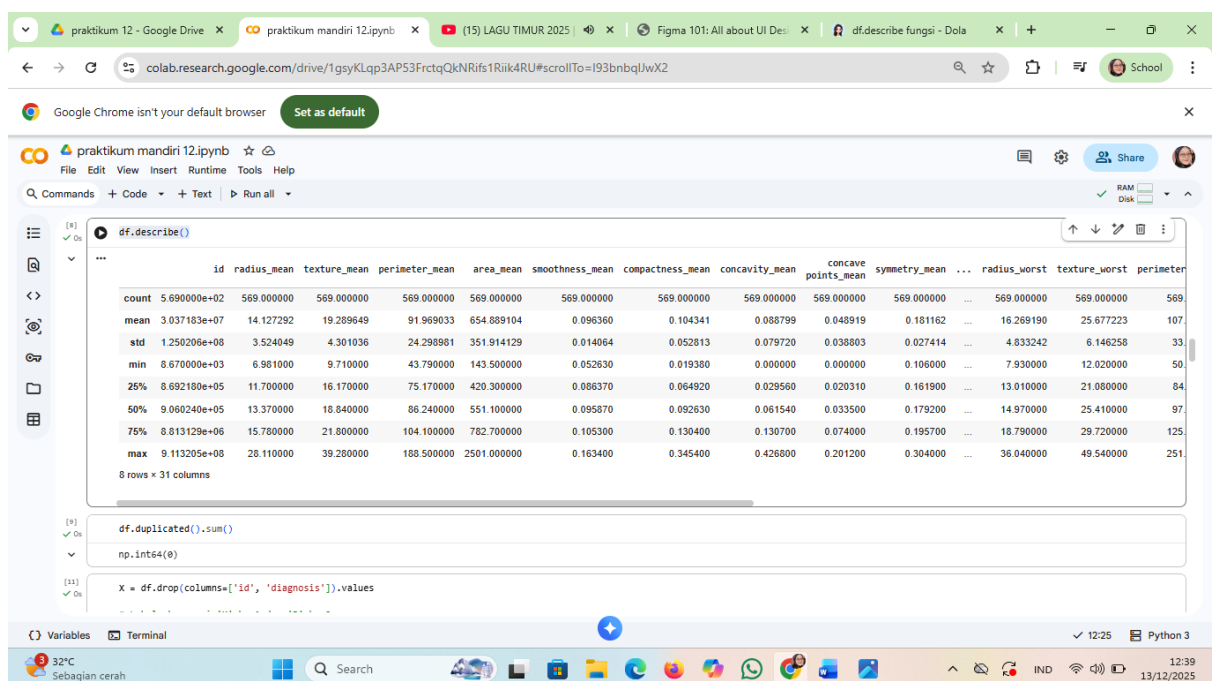
5. digunakan untuk menghitung jumlah nilai yang hilang (null) pada setiap kolom dalam sebuah DataFrame.



```
[7]: df.isnull().sum()
```

radius_mean	0
texture_mean	0
perimeter_mean	0
area_mean	0
smoothness_mean	0
compactness_mean	0
concavity_mean	0
concave points_mean	0
symmetry_mean	0
fractal_dimension_mean	0
radius_se	0
texture_se	0
perimeter_se	0
area_se	0
smoothness_se	0
compactness_se	0

6. df.describe() digunakan untuk menghasilkan statistik deskriptif dari sebuah DataFrame pada library Pandas di Python.



```
[8]: df.describe()
```

	id	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	symmetry_mean	...	radius_worst	texture_worst	perimeter
count	5.690000e+02	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	...	569.000000	569.000000	569
mean	3.037183e+07	14.127292	19.289649	91.969033	654.889104	0.096360	0.104341	0.088799	0.048919	0.181162	...	16.269190	25.677223	107
std	1.250206e+08	3.524049	4.301036	24.298981	351.914129	0.014064	0.052813	0.079720	0.038803	0.027414	...	4.833242	6.146258	33
min	8.670000e+03	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380	0.000000	0.000000	0.106000	...	7.930000	12.020000	50
25%	8.692180e+05	11.700000	16.170000	75.170000	420.300000	0.086370	0.064920	0.029560	0.020310	0.161900	...	13.010000	21.080000	84
50%	9.060240e+05	13.370000	18.840000	86.240000	551.100000	0.095870	0.092630	0.061540	0.033500	0.179200	...	14.970000	25.410000	97
75%	8.813129e+06	15.780000	21.800000	104.100000	762.700000	0.105300	0.130400	0.130700	0.074000	0.195700	...	18.790000	29.720000	125
max	9.113205e+08	28.110000	39.280000	188.500000	2501.000000	0.163400	0.345400	0.426800	0.201200	0.304000	...	36.040000	49.540000	251

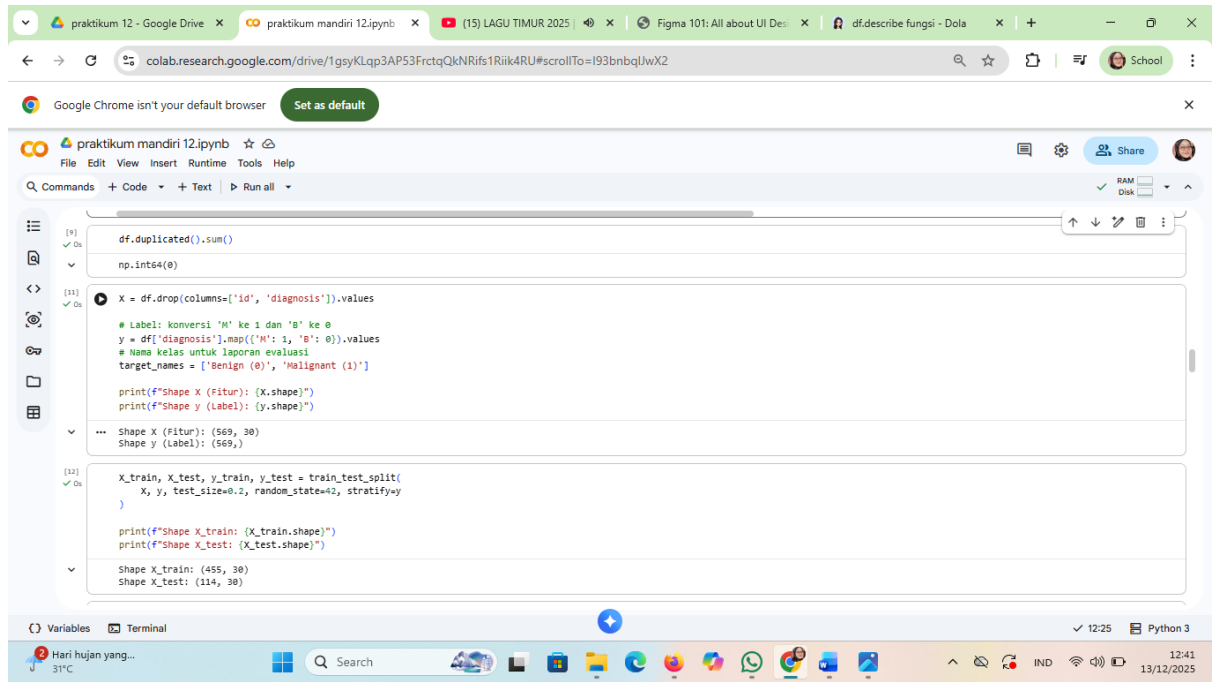
8 rows x 31 columns

```
[9]: df.duplicated().sum()
```

```
np.int64(0)
```

```
[11]: x = df.drop(columns=['id', 'diagnosis']).values
```

7. , kode ini melakukan persiapan data untuk machine learning, termasuk menghapus kolom yang tidak diperlukan, mengubah label kategorikal menjadi numerik, dan membagi data menjadi data latih dan data uji.



The screenshot shows a Google Colab notebook titled "praktikum mandiri 12.ipynb". The notebook contains three code cells. The first cell runs `df.duplicated().sum()` and `np.int64(0)`. The second cell drops columns 'id' and 'diagnosis' from the dataframe, converts the 'diagnosis' column to numerical values (0 for 'B' and 1 for 'M'), and prints the shapes of the resulting X and y arrays. The third cell splits the data into training and testing sets using `train_test_split` and prints the shapes of the resulting X_train, X_test, y_train, and y_test arrays.

```
[9] ✓ On
df.duplicated().sum()
np.int64(0)

[11] ✓ On
X = df.drop(columns=['id', 'diagnosis']).values
# Label: konversi 'M' ke 1 dan 'B' ke 0
y = df['diagnosis'].map({'M': 1, 'B': 0}).values
# Nama kelas untuk laporan evaluasi
target_names = ['Benign (0)', 'Malignant (1)']

print(f"Shape X (Fitur): {X.shape}")
print(f"Shape y (Label): {y.shape}")

*** Shape X (Fitur): (569, 30)
Shape y (Label): (569,)

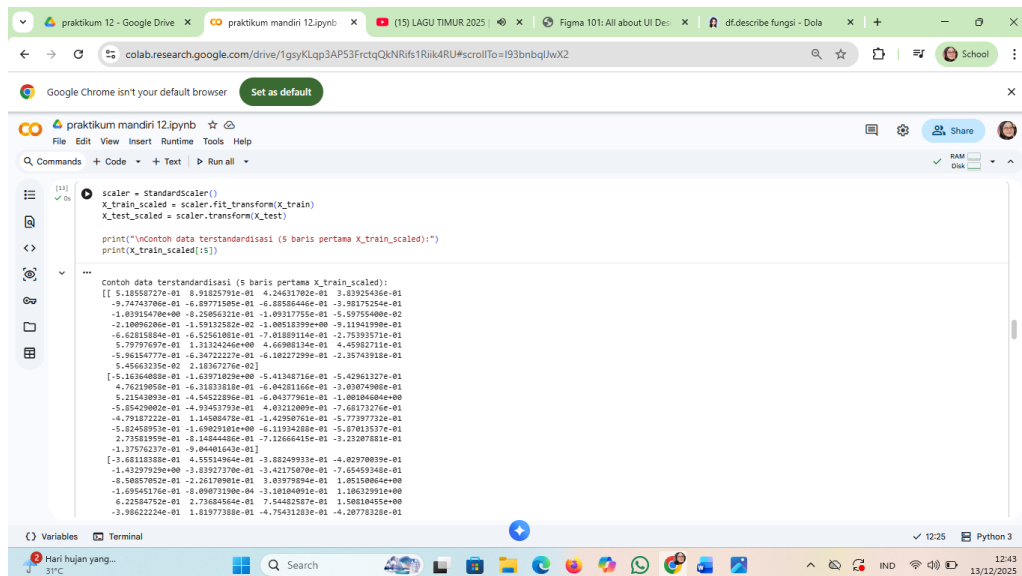
[12] ✓ On
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"Shape X_train: {X_train.shape}")
print(f"Shape X_test: {X_test.shape}")

Shape X_train: (455, 30)
Shape X_test: (114, 30)
```

Variables Terminal Python 3 12:25 13/12/2025

8. standarisasi pada data latih dan data uji menggunakan StandardScaler. Standarisasi adalah proses mengubah data sehingga memiliki mean 0 dan standar deviasi 1. Hal ini penting karena banyak algoritma machine learning bekerja lebih baik ketika fitur-fitur memiliki skala yang sama.



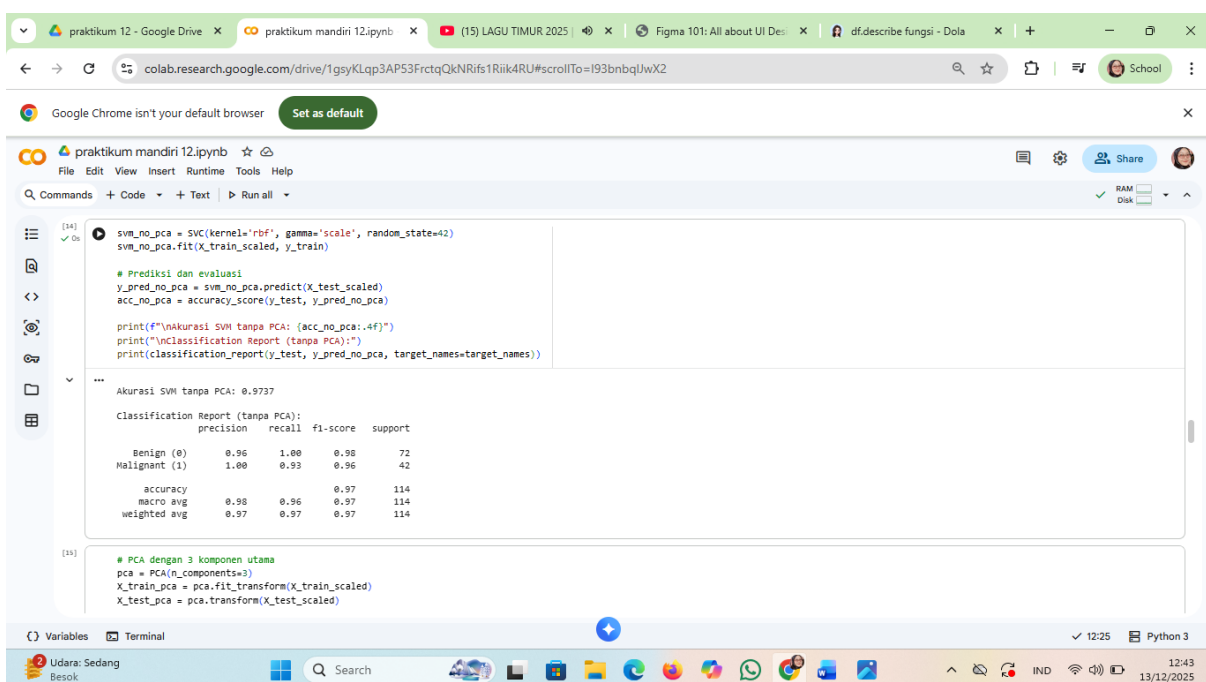
```
[21]: scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nContoh data terstandarisasi (5 baris pertama X_train_scaled):")
print(X_train_scaled[:5])
```

Contoh data terstandarisasi (5 baris pertama X_train_scaled):

```
[[ 5.18558727e-01  8.9125791e-01  4.24631702e-01  3.83925436e-01
 -9.74743706e-01 -6.69771505e-01 -6.8838644e-01 -3.98175254e-01
 -1.89154780e+00 -6.2566521e-01 -1.0817755e-01 -5.59755400e-02
 -2.10896286e-01 -1.59132582e-02 -1.08518399e+00 -9.11941990e-01
 -6.6231584e-01 -6.52561801e-01 -7.01889114e-01 -2.75393571e-01
 5.79797597e-01 1.31324246e+00 4.6698134e-01 4.45982711e-01
 5.96154777e-01 -6.34722227e-01 -6.18227299e-01 -2.35743918e-01
 5.45663225e-02 2.18367276e-02]
[-5.16364886e-01 -1.63971829e+00 -5.41348716e-01 -5.42961327e-01
 4.76219859e-01 -6.31833818e-01 -6.04281164e-01 -3.03074908e-01
 5.21543893e-01 -4.54522896e-01 -6.04377961e-01 -1.00104040e+00
 -5.85429062e-01 -4.93453793e-01 4.03212089e-01 -7.68173276e-01
 -4.79187322e-01 1.14508478e-01 -1.42950761e-01 -5.77397732e-01
 -5.82458953e-01 -1.69629331e+00 -6.11934288e-01 -5.87813537e-01
 2.73581959e-01 -8.14544486e-01 -7.12666415e-01 -3.23287881e-01
 -1.37576237e-01 -9.04401643e-01]
[-3.68118386e-01 4.55514944e-01 -3.88249931e-01 -4.02978003e-01
 -1.43297929e+00 -3.83927370e-01 -3.42175070e-01 -7.65459348e-01
 -8.58857852e-01 -2.26178981e-01 3.83979894e-01 1.05158864e+00
 -1.69545176e-01 -8.49873190e-04 -3.18184891e-01 -1.18632291e+00
 6.22584752e-01 2.73684564e-01 7.54482587e-01 1.58818455e+00
 -3.98622246e-01 1.81977388e-01 -4.75431283e-01 -4.28778328e-01]
```

9. model SVM dengan kernel RBF pada data yang telah diskalakan, lalu mengevaluasi performa model menggunakan akurasi dan laporan klasifikasi. Model ini dilatih tanpa menggunakan PCA untuk reduksi dimensi.



```
[14]: svm_no_pca = SVC(kernel='rbf', gamma='scale', random_state=42)
svm_no_pca.fit(X_train_scaled, y_train)

# Prediksi dan evaluasi
y_pred_no_pca = svm_no_pca.predict(X_test_scaled)
acc_no_pca = accuracy_score(y_test, y_pred_no_pca)

print(f"Akurasi SVM tanpa PCA: {acc_no_pca:.4f}")
print("\nClassification Report (tanpa PCA):")
print(classification_report(y_test, y_pred_no_pca, target_names=target_names))
```

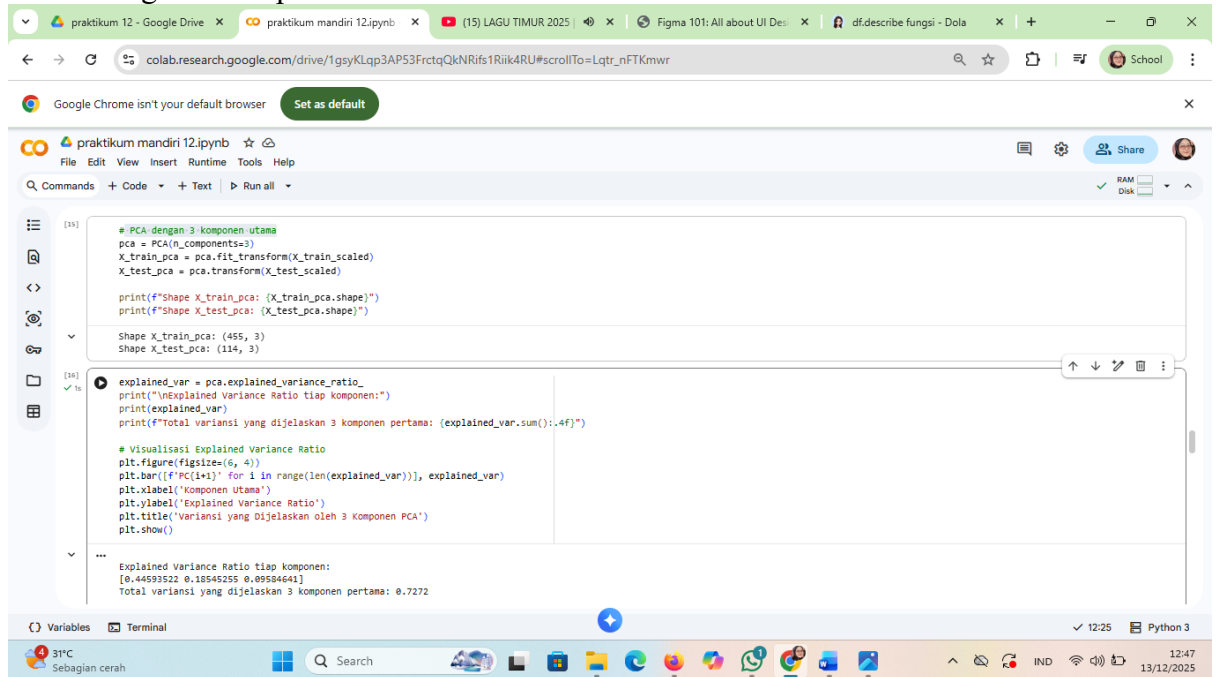
Akurasi SVM tanpa PCA: 0.9737

Classification Report (tanpa PCA):

	precision	recall	f1-score	support
Benign (0)	0.96	1.00	0.98	72
Malignant (1)	1.00	0.93	0.96	42
accuracy			0.97	114
macro avg	0.98	0.96	0.97	114
weighted avg	0.97	0.97	0.97	114

```
[15]: # PCA dengan 3 komponen utama
pca = PCA(n_components=3)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)
```

10. PCA dengan 3 komponen utama



The screenshot shows a Jupyter Notebook interface with the following code and output:

```
[15]: # PCA dengan 3 komponen utama
pca = PCA(n_components=3)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

print(f'Shape X_train_pca: {X_train_pca.shape}')
print(f'Shape X_test_pca: {X_test_pca.shape}')

Shape X_train_pca: (455, 3)
Shape X_test_pca: (114, 3)

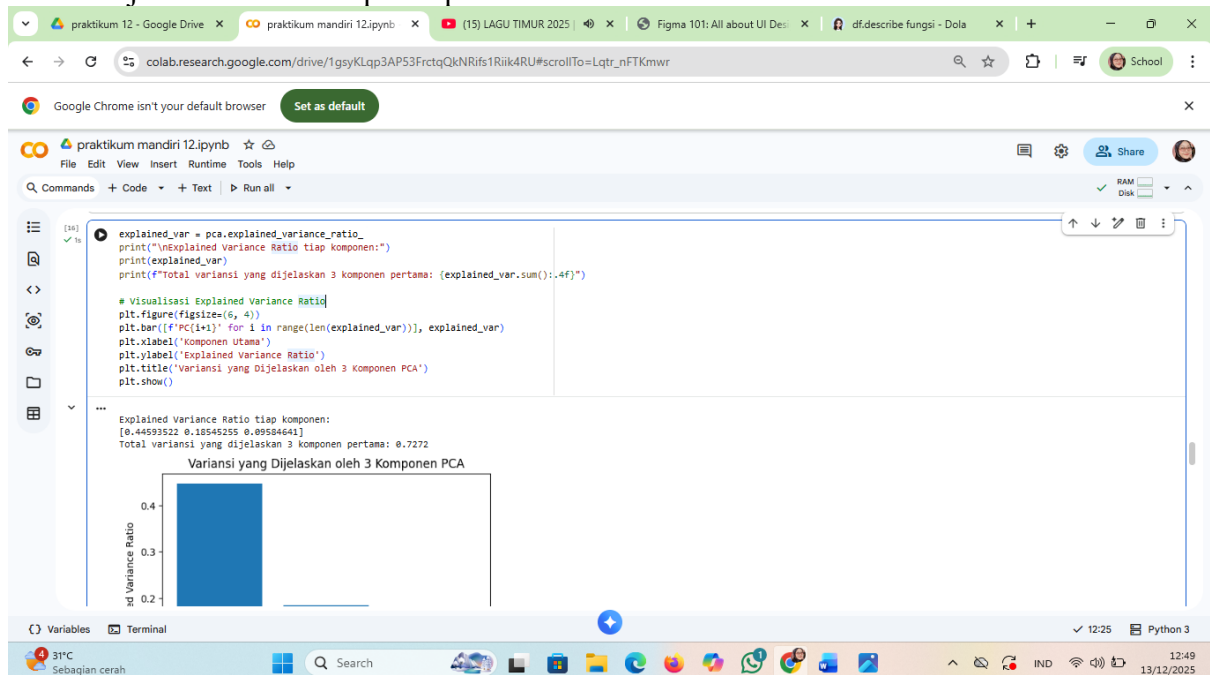
[16]: explained_var = pca.explained_variance_ratio_
print("\nExplained Variance Ratio tiap komponen:")
print(explained_var)
print(f'Total variansi yang dijelaskan 3 komponen pertama: {explained_var.sum():.4f}')

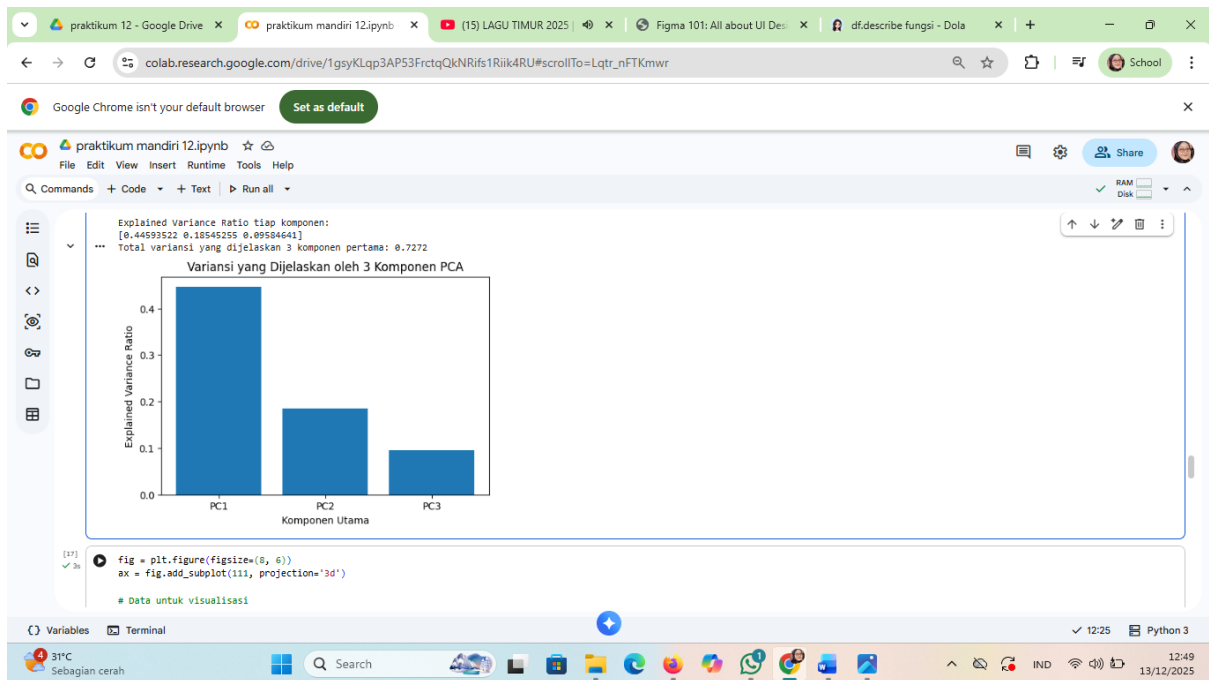
# Visualisasi Explained Variance Ratio
plt.figure(figsize=(6, 4))
plt.bar([f'PC{i+1}' for i in range(len(explained_var))], explained_var)
plt.xlabel('Komponen Utama')
plt.ylabel('Explained Variance Ratio')
plt.title('Variansi yang Dijelaskan oleh 3 Komponen PCA')
plt.show()
```

Output:

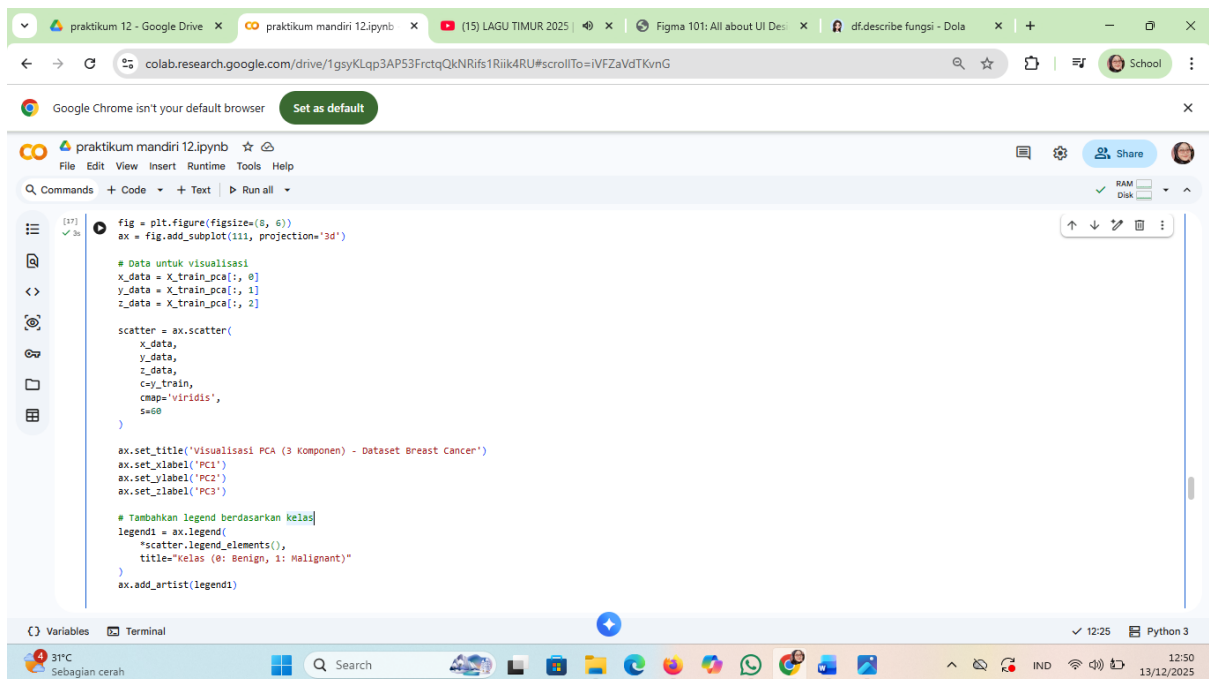
```
Explained Variance Ratio tiap komponen:
[0.44593522 0.18545255 0.89584641]
Total variansi yang dijelaskan 3 komponen pertama: 0.7272
```

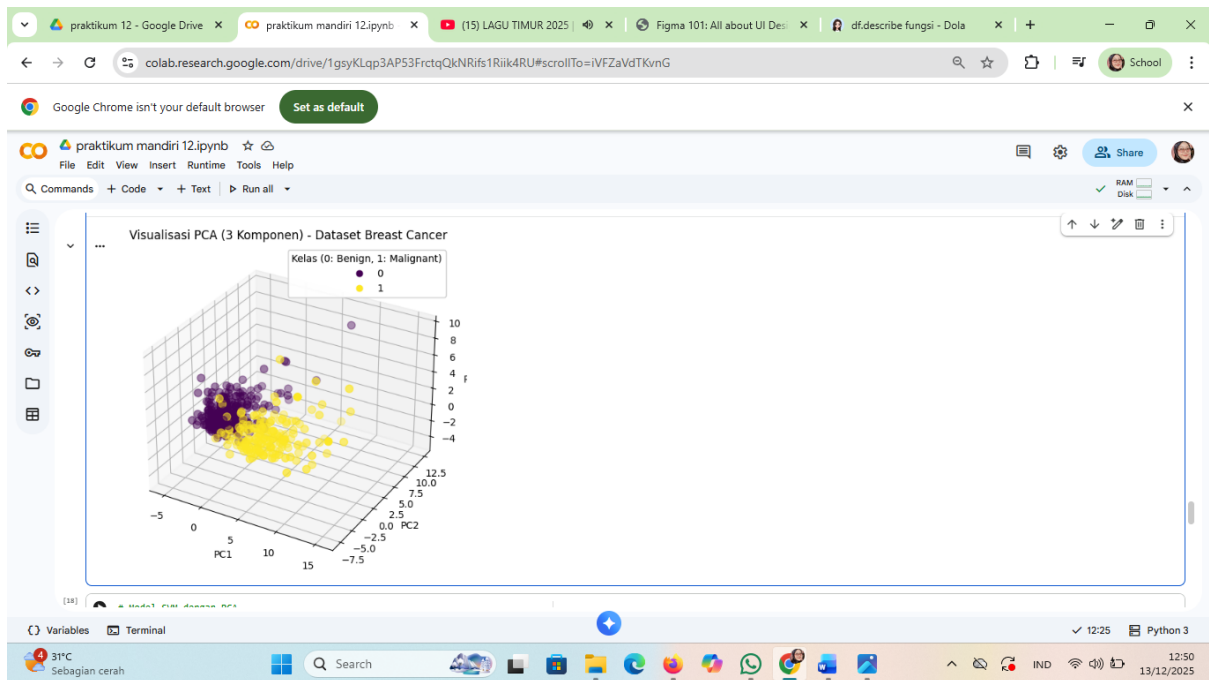
11. menganalisis hasil PCA dengan mencetak proporsi varians yang dijelaskan oleh setiap komponen utama dan total varians yang dijelaskan oleh semua komponen. Selain itu, kode ini juga membuat bar plot untuk memvisualisasikan proporsi varians yang dijelaskan oleh setiap komponen utama.



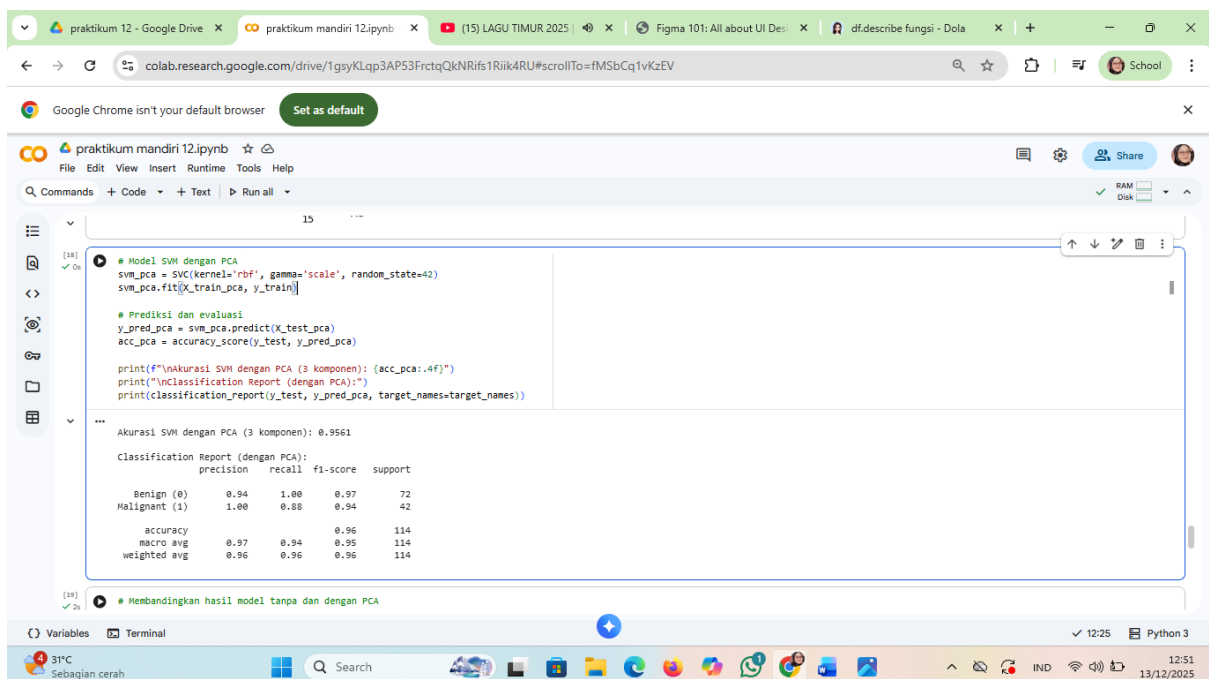


12. membuat visualisasi 3D dari data yang telah direduksi dimensinya menjadi 3 komponen utama menggunakan PCA. Plot ini memungkinkan untuk melihat bagaimana data terdistribusi dalam ruang 3D yang dibentuk oleh komponen-komponen utama, dan bagaimana kelas-kelas dalam data dipisahkan dalam ruang ini.





13. model SVM dengan kernel RBF pada data yang telah direduksi dimensinya menggunakan PCA (menjadi 3 komponen), lalu mengevaluasi performa model menggunakan akurasi dan laporan klasifikasi. Hasil evaluasi (akurasi dan laporan klasifikasi) dicetak ke konsol. Kode ini memungkinkan untuk membandingkan performa model SVM dengan dan tanpa PCA untuk melihat apakah reduksi dimensi meningkatkan atau menurunkan akurasi model.



14. membandingkan performa model SVM dengan dan tanpa PCA dalam bentuk tabel dan visualisasi. Tabel menunjukkan jumlah fitur, akurasi, dan total variansi yang dijelaskan oleh PCA (jika digunakan). Bar plot memvisualisasikan perbandingan akurasi antara kedua model, dengan nilai akurasi ditampilkan di atas setiap bar. Ini memungkinkan untuk dengan mudah melihat apakah PCA meningkatkan atau menurunkan akurasi model, dan seberapa besar perbedaannya.

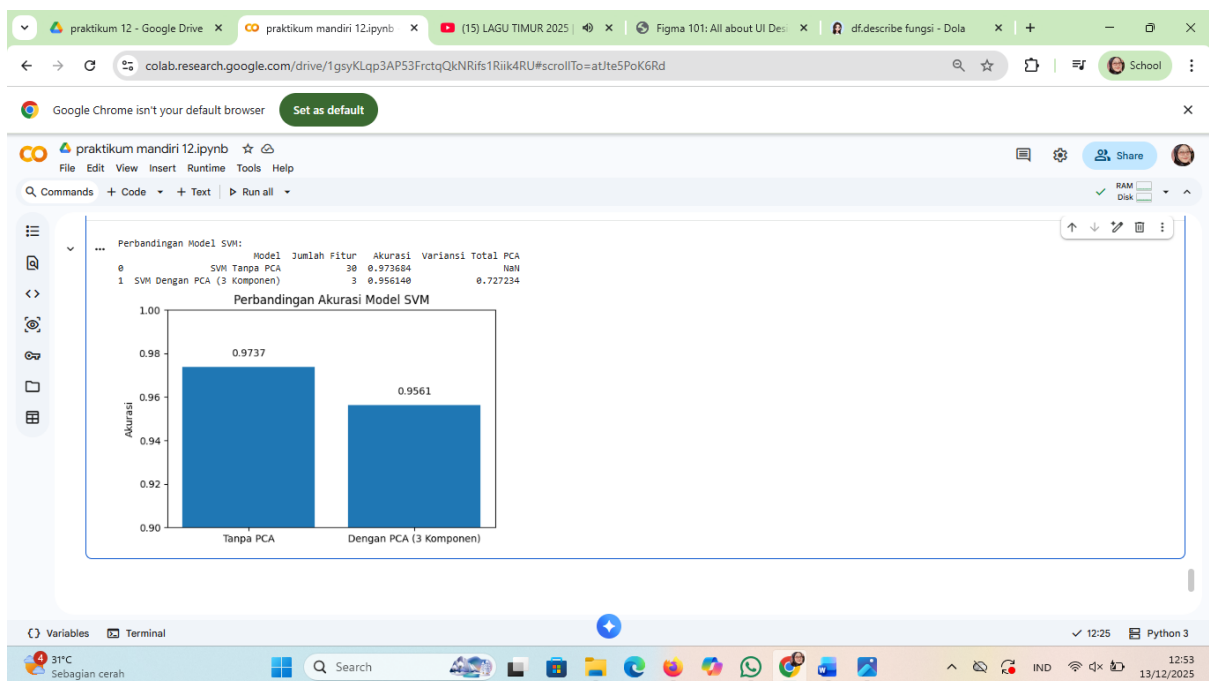
```
macro avg    0.97    0.94    0.95    114
weighted avg 0.96    0.96    0.96    114

# Membandingkan hasil model tanpa dan dengan PCA
comparison = pd.DataFrame({
    'Model': ['SVM Tanpa PCA', 'SVM Dengan PCA (3 Komponen)'],
    'Jumlah Fitur': [X_train_scaled.shape[1], X_train_pca.shape[1]],
    'Akurasi': [acc_no_pca, acc_pca],
    'Variansi Total PCA': [None, explained_var.sum()]
})
print("\nPerbandingan Model SVM:")
print(comparison)

# Visualisasi perbandingan akurasi
plt.figure(figsize=(6, 4))
models = ['Tanpa PCA', 'Dengan PCA (3 Komponen)']
accuracies = [acc_no_pca, acc_pca]
plt.bar(models, accuracies)
plt.title('Perbandingan Akurasi Model SVM')
plt.xlabel('Akurasi')
plt.ylim(0.9, 1.0) # Fokus pada rentang 0.9 sampai 1.0

# Menambahkan nilai akurasi di atas bar
for i, v in enumerate(accuracies):
    plt.text(i, v + 0.005, f'{v:.4f}', ha='center')

plt.show()
```



Referensi:

- Munir, S., Seminar, K. B., Sudradjat, Sukoco, H., & Buono, A. (2022). The Use of Random Forest Regression for Estimating Leaf Nitrogen Content of Oil Palm Based on Sentinel 1-A Imagery. *Information*, 14(1), 10. <https://doi.org/10.3390/info14010010>
- Seminar, K. B., Imantho, H., Sudradjat, Yahya, S., Munir, S., Kaliana, I., Mei Haryadi, F., Noor Baroroh, A., Supriyanto, Handoyo, G. C., Kurnia Wijayanto, A., Ijang Wahyudin, C., Liyantono, Budiman, R., Bakir Pasaman, A., Rusiawan, D., & Sulastri. (2024). PreciPalm: An Intelligent System for Calculating Macronutrient Status and Fertilizer Recommendations for Oil Palm on Mineral Soils Based on a Precision Agriculture Approach. *Scientific World Journal*, 2024(1). <https://doi.org/10.1155/2024/1788726>

LINK GITHUB: <https://github.com/Sitiaisah1604/machine-learning>